Runbook: PowerShell (Windows)

Every command to build the Health-style lab Genesys Cloud lab from a Windows PC. One action per step. After each step there is **You should see**.

If you prefer Bash, use `RUNBOOK-BASH.md`. The two do the same thing.

Before you start: read Part 1 and 2 of `BUILD-MANUAL.md` (a lab org and an OAuth client). **Never run this against a customer's org.**

Works in Windows PowerShell 5.1 and PowerShell 7. Use PowerShell 7 if you can.

Step 1. Install the tools (once)

Open **PowerShell**. Paste these one at a time. `winget` is built into current Windows.

```
winget install --id Git.Git -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id OpenTofu.Tofu -e
```

Close PowerShell and open it again so the new tools are found. Then check:

```
git --version
node --version
tofu version
```

You should see a version number for each. Node should be 22 or newer. If `node` is older, install the current LTS.

> Prefer HashiCorp Terraform? Install it instead of OpenTofu (`winget install --id Hashicorp.Terraform -e`). The scripts use whichever they find, OpenTofu first.

Step 2. Allow scripts to run (once)

Windows blocks scripts by default. This allows scripts **for this window only**:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

You should see nothing (that means it worked). You must run it again in every new window.

Step 3. Go to the repo and set your credentials

Change the path to where you put the repo.

```
cd "$HOME\Downloads\GitHub\genesys-health-style-lab"
```

Set the three variables. Paste your own values. **They exist only in this window.**

```
$env:GENESYSCLOUD_OAUTHCLIENT_ID     = "paste-client-id-here"
$env:GENESYSCLOUD_OAUTHCLIENT_SECRET = "paste-secret-here"
$env:GENESYSCLOUD_REGION             = "ap-southeast-2"
```

> Do not save these in a file. Do not type them into a script. Do not paste them into chat.

You should see no output. Check they are set without showing the secret:

```
[bool]$env:GENESYSCLOUD_OAUTHCLIENT_SECRET
```

It prints `True`.

Step 4. Preflight (changes nothing)

```
.\scripts\powershell\00-preflight.ps1
```

You should see: `OK node found`, `OK IaC tool found`, `OK Token received for region ap-southeast-2`, and `OK Connected to Genesys org: <your org name>`.

Check the org name is your lab org. If it is not, close the window.

If you see	Do this
`Set GENESYSCLOUD_OAUTHCLIENT_ID`	Step 3 was not done in this window
`Could not get a token`	Wrong client ID, secret or region
`Neither tofu nor terraform is installed`	Step 1, then reopen PowerShell

Step 5. Package the function

```
.\scripts\powershell\01-package-function.ps1
```

You should see: `# tests 8`, `# pass 8`, `# fail 0`, `OK Built ...normalise-reference.zip`.

Step 6. Initialise and validate

```
.\scripts\powershell\02-init.ps1
```

You should see: `OK Formatting is clean`, then `Success! The configuration is valid.`, then `OK Configuration is valid`.

If formatting fails, run `tofu fmt -recursive` inside the `terraform` folder, and run Step 6 again.

After this step a file `terraform\.terraform.lock.hcl` exists. **Keep it and commit it later.**

Step 6b. The offline plan test (needs no org)

This plans the whole design four ways (minimal, full, surge switch on, sign-in hardening on) with the Genesys provider **mocked**. It needs no credentials and changes nothing. It catches mistakes that `validate` cannot see, such as a queue with too few rings or a percentage given as 70 instead of 0.70.

```
cd terraform
tofu init -backend=false
tofu test
cd ..
```

You should see `Success! 4 passed, 0 failed.` OpenTofu 1.8 or Terraform 1.7 or newer is needed.

> It proves the design expands into objects the provider will accept. It does **not** prove Genesys will accept them. Only a real `apply` proves that.

Step 7. Plan (changes nothing)

Smaller build (no AI, quality or web chat), a good first try:

```
.\scripts\powershell\03-plan.ps1 -Environment minimal
```

Full build:

```
.\scripts\powershell\03-plan.ps1 -Environment full
```

Full build plus sign-in hardening (**lab org only**; read `terraform\modules\security\main.tf` first, because a wrong setting can lock everyone out, including you, so keep a second admin session open):

```
.\scripts\powershell\03-plan.ps1 -Environment hardened
```

You should see a long list and a line like `Plan: N to add, 0 to change, 0 to destroy.`

Read the plan. Names should start with `DH`. On a first run, nothing should be changed or destroyed.

If the plan fails, read the **last** error. The most common first-run errors are in the table at the end.

Step 8. Apply (this changes Genesys)

```
.\scripts\powershell\04-apply.ps1 -Environment minimal
```

(or `-Environment full`). It asks you to type `YES`.

You should see `Apply complete! Resources: N added.`

If it fails partway: read the error, fix the code, and run Steps 7 and 8 again. Terraform adds only what is missing.

Step 9. Inventory

```
.\scripts\powershell\05-inventory.ps1
```

You should see counts of objects, then a list of queues beginning `DH`.

Now open Genesys Cloud and look at what was built (see `BUILD-MANUAL.md`, "Look at what you built"). Then run the tests in the Manual, Part 6.

Step 9b. Health check (read-only)

```
.\scripts\powershell\08-health-check.ps1
```

You should see five numbered checks and `OK All checks passed`. If a flow is saved but not published, or an expected queue is missing, it prints `FAIL` and exits with 1, so a scheduler can alert on it. A `WARN` on the surge

switch means someone left it on.

It was tested against a mock server only. If a field name differs in your org, fix the script and note it in your run log.

Step 10. The surge drill

Open `terraform\envs\full.tfvars` in Notepad. Remove the `#` from the line `surge_mode_enabled = true`. Save. Then:

```
.\scripts\powershell\03-plan.ps1 -Environment full
```

You should see one change: the emergency group `enabled`. Then:

```
.\scripts\powershell\04-apply.ps1 -Environment full
```

Put the `#` back, plan, apply, to switch it off.

Step 11. Optional: read the org back as Terraform

```
.\scripts\powershell\06-export.ps1
```

You should see `OK Exported to ...\export\generated`. Open the files, look, and do not commit them: they contain real IDs. The `export` folder is already in `.gitignore`.

Step 12. Tear down

```
.\scripts\powershell\05-inventory.ps1
.\scripts\powershell\07-destroy.ps1
.\scripts\powershell\05-inventory.ps1
```

The destroy script shows what it will delete. Type `DESTROY` to continue. **You should see** `OK Destroyed.` and, in the final inventory, no `DH` queues.

Then check **Admin, Billing** in Genesys and your contract for numbers, add-ons and usage that remain.

Step 13. Put it on GitHub

First run the safety check:

```
pip install pyyaml python-hcl2
python tools\check_repo.py
```

You should see `All checks passed.` (If `python` is not found, try `py tools\check_repo.py`.)

Then:

```
git init -b main
git add .
git status
```

Read the `git status` list. **You must not see** `.tfstate`, `.tfplan`, `.env`, or a `backend.tf`. If you do, stop and fix `.gitignore`.

```
git commit -m "Initial commit: Health-style lab"
git remote add origin https://github.com/asvinluan/genesys-health-style-lab.git
git push -u origin main
```

You should see your files on GitHub and, within a minute or two, the **Actions** tab running the checks.

Common errors

What you see	Meaning	Fix
`running scripts is disabled on this system`	Step 2 was not done	Run Step 2 in this window
`Set GENESYSCLOUD_OAUTHCLIENT_ID`	Variables not set in this window	Step 3
`Could not get a token`	Wrong ID, secret or region	Re-check Part 2 of the Manual
`Error: 403` or `missing permission`	The OAuth client's role is too narrow	Add the permission named in the error
License or "feature not enabled"	Org lacks AI, WFM, quality or knowledge	Use `-Environment minimal`, or set `enable_ai = false` etc. in a tfvars
`already exists`	A previous run left an object	Delete it in Genesys, or import it
Plan wants to destroy something	Code and org disagree	Do not apply. Find out why first.
`Files are not formatted`	Formatting only	`tofu fmt -recursive` in `terraform`
`No module named hcl2`	The checker's two helpers are missing	`pip install pyyaml python-hcl2`
`python is not recognized`	Python not installed	`winget install --id Python.Python.3.12 -e`, reopen PowerShell

What each script does

Script	Purpose	Changes Genesys
`00-preflight.ps1`	Tools, token, org name	No
`01-package-function.ps1`	Test and zip the function	No
`02-init.ps1`	Format check, init, validate	No
`03-plan.ps1`	Show what would be built	No
`04-apply.ps1`	Build from the saved plan	**Yes**
`05-inventory.ps1`	Count lab objects	No
`06-export.ps1`	Read the org back as Terraform	No
`07-destroy.ps1`	Delete what this repo built	**Yes**

Script	Purpose	Changes Genesys
`08-health-check.ps1`	Read-only check that what was built is still healthy	No
`lib.ps1`	Shared helpers, not run directly	No

All scripts stop at the first failure. They never print the secret or the token.